PYTHON PROGRAMMING
LAB RECORD

College Practical Record | Python Case Studies

Name: _____

Roll Number: _____

Department / Section: _____

Academic Year: _____

Prepared from the uploaded case-study PDF. Source code is reproduced from extracted PDF text; original .py files were not provided.

TABLE OF CONTENTS

EXPERIMENT 01: Student Marks and Result Analysis 3

EXPERIMENT 02: Bank Transaction and Balance Management System 6

EXPERIMENT 03: Bus Route Capacity and Passenger Demand Management 9

EXPERIMENT 04: Traffic Violation Penalty Calculator 11

EXPERIMENT 05: Campus Event Financial Planning 14

EXPERIMENT 06: Employee Payroll and Deduction Analysis 17

EXPERIMENT 07: Campus Festival Registration and Financial Audit 20

EXPERIMENT 08: Water Distribution Fairness Analysis 23

EXPERIMENT 09: Mobile Plan Cost Optimization System 26

EXPERIMENT 10: Food Waste Monitoring System 29

EXPERIMENT 01: Student Marks and Result Analysis

1. Aim

To develop and examine a Python program for student marks and result analysis, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

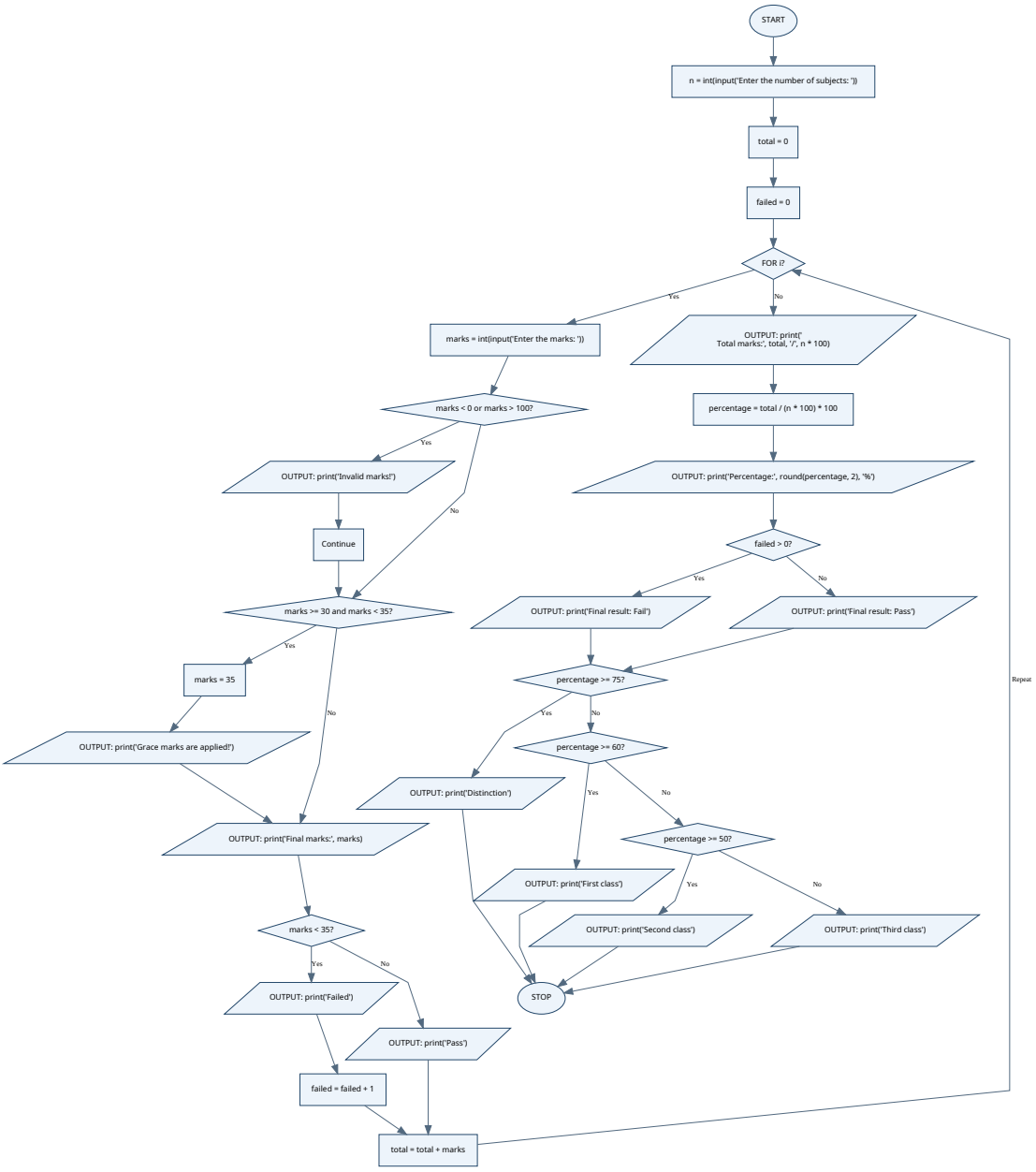
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
n = int(input('Enter the number of subjects: '))
total = 0
failed = 0
FOR i
  EXECUTE loop body
END LOOP
OUTPUT print('\nTotal marks:', total, '/', n * 100)
percentage = total / (n * 100) * 100
OUTPUT print('Percentage:', round(percentage, 2), '%')
IF failed > 0
  EXECUTE conditional branch
ELSE (if applicable)
END IF
IF percentage >= 75
  EXECUTE conditional branch
ELSE (if applicable)
END IF
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
n = int(input("Enter the number of subjects: "))
total = 0
failed = 0
for i in range(1, n + 1):
    marks = int(input("Enter the marks: "))
    if marks < 0 or marks > 100:
        print("Invalid marks!")
        continue
    if marks >= 30 and marks < 35:
        marks = 35
        print("Grace marks are applied!")
    print("Final marks:", marks)
    if marks < 35:
        print("Failed")
        failed = failed + 1
    else:
        print("Pass")
    total = total + marks
print("\nTotal marks:", total, "/", n * 100)
percentage = (total / (n * 100)) * 100
print("Percentage:", round(percentage, 2), "%")
if failed > 0:
    print("Final result: Fail")
else:
    print("Final result: Pass")
if percentage >= 75:
    print("Distinction")
elif percentage >= 60:
    print("First class")
elif percentage >= 50:
    print("Second class")
else:
    print("Third class")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['3', '80', '75', '90']

```
Enter the number of subjects: Enter the marks: Final marks: 80
Pass
Enter the marks: Final marks: 75
Pass
Enter the marks: Final marks: 90
Pass
```

```
Total marks: 245 / 300
Percentage: 81.67 %
Final result: Pass
Distinction
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 3, 80, 75, 90	Defined by original program logic; see actual execution	Final result: Pass; Distinction	Executed
TC02	Boundary / alternate input 2, 32, 70	Defined by original program logic; see actual execution	Final result: Pass; Second class	Executed
TC03	Invalid / edge input 1, -1	Defined by original program logic; see actual execution	Final result: Pass; Third class	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 02: Bank Transaction and Balance Management System

1. Aim

To develop and examine a Python program for bank transaction and balance management system, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

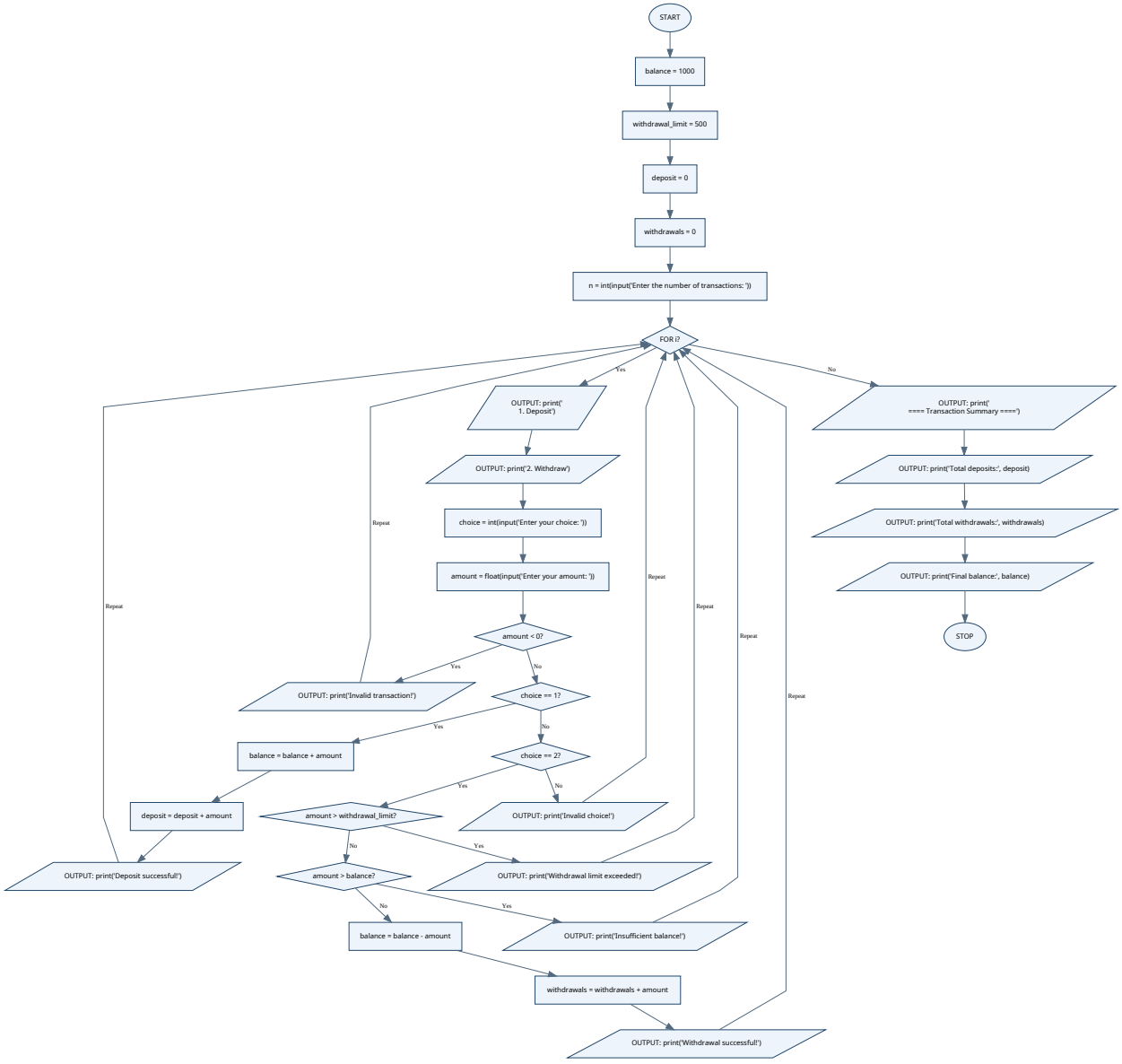
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
balance = 1000
withdrawal_limit = 500
deposit = 0
withdrawals = 0
n = int(input('Enter the number of transactions: '))
FOR i
  EXECUTE loop body
END LOOP
OUTPUT print('\n==== Transaction Summary ====')
OUTPUT print('Total deposits:', deposit)
OUTPUT print('Total withdrawals:', withdrawals)
OUTPUT print('Final balance:', balance)
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
balance = 1000
withdrawal_limit = 500
deposit = 0
withdrawals = 0
n = int(input("Enter the number of transactions: "))
for i in range(n):
    print("\n1. Deposit")
    print("2. Withdraw")
    choice = int(input("Enter your choice: "))
    amount = float(input("Enter your amount: "))
    if amount < 0:
        print("Invalid transaction!")
    elif choice == 1:
        balance = balance + amount
        deposit = deposit + amount
        print("Deposit successful!")
    elif choice == 2:
        if amount > withdrawal_limit:
            print("Withdrawal limit exceeded!")
        elif amount > balance:
            print("Insufficient balance!")
        else:
            balance = balance - amount
            withdrawals = withdrawals + amount
            print("Withdrawal successful!")
    else:
        print("Invalid choice!")
print("\n==== Transaction Summary ====")
print("Total deposits:", deposit)
print("Total withdrawals:", withdrawals)
print("Final balance:", balance)
```

7. Output (Executed Sample)

Sample input values (in entry order): ['2', '1', '500', '2', '300']

```
Enter the number of transactions:
1. Deposit
2. Withdraw
Enter your choice: Enter your amount: Deposit successful!
1. Deposit
2. Withdraw
Enter your choice: Enter your amount: Withdrawal successful!
==== Transaction Summary ====
Total deposits: 500.0
Total withdrawals: 300.0
Final balance: 1200.0
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 2, 1, 500, 2, 300	Defined by original program logic; see actual execution	Total withdrawals: 300.0; Final balance: 1200.0	Executed
TC02	Boundary / alternate input 1, 2, 500	Defined by original program logic; see actual execution	Total withdrawals: 500.0; Final balance: 500.0	Executed
TC03	Invalid / edge input 1, 2, 600	Defined by original program logic; see actual execution	Total withdrawals: 0; Final balance: 1000	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 03: Bus Route Capacity and Passenger Demand Management

1. Aim

To develop and examine a Python program for bus route capacity and passenger demand management, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

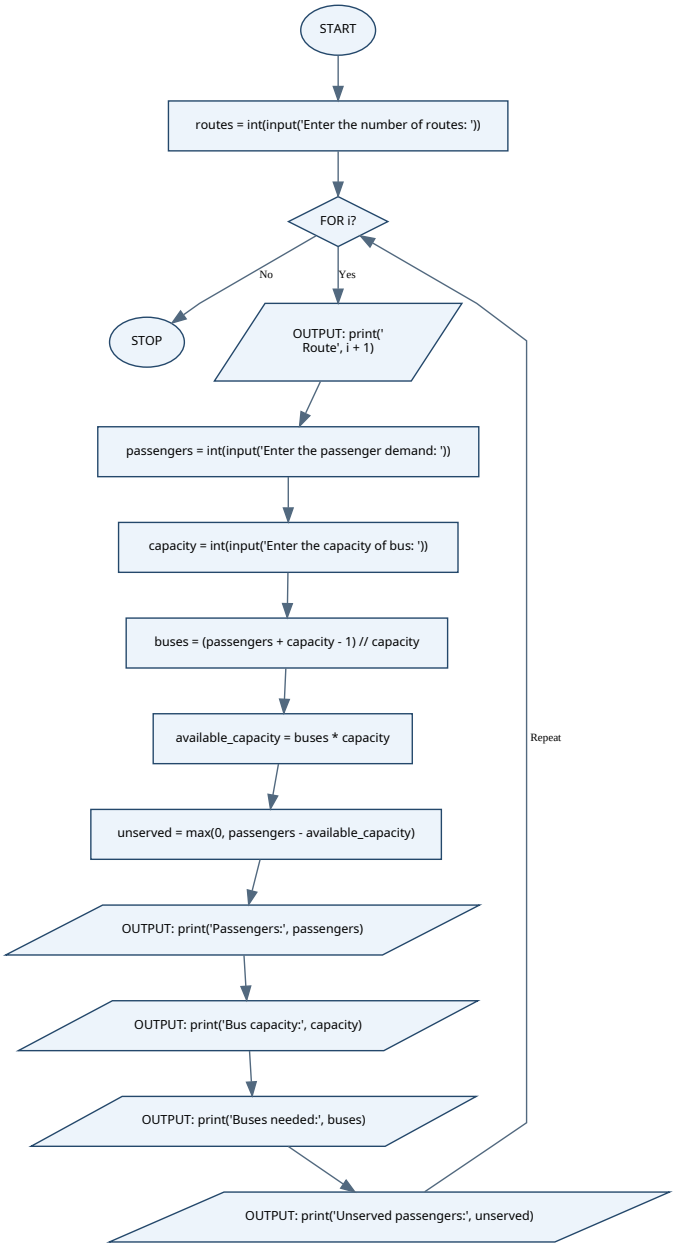
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
routes = int(input('Enter the number of routes: '))
FOR i
  EXECUTE loop body
END LOOP
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
routes = int(input("Enter the number of routes: "))
for i in range(routes):
    print("\nRoute", i + 1)
    passengers = int(input("Enter the passenger demand: "))
    capacity = int(input("Enter the capacity of bus: "))
    buses = (passengers + capacity - 1) // capacity
    available_capacity = buses * capacity
    unserved = max(0, passengers - available_capacity)
    print("Passengers:", passengers)
    print("Bus capacity:", capacity)
    print("Buses needed:", buses)
    print("Unserved passengers:", unserved)
```

7. Output (Executed Sample)

Sample input values (in entry order): ['1', '95', '40']

```
Enter the number of routes:
Route 1
Enter the passenger demand: Enter the capacity of bus: Passengers: 95
Bus capacity: 40
Buses needed: 3
Unserved passengers: 0
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 1, 95, 40	Defined by original program logic; see actual execution	Buses needed: 3; Unserved passengers: 0	Executed
TC02	Boundary / alternate input 1, 0, 40	Defined by original program logic; see actual execution	Buses needed: 0; Unserved passengers: 0	Executed
TC03	Invalid / edge input 1, 20, 0	Defined by original program logic; see actual execution	~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^~~~~~~~~~; ZeroDivisionError: integer division or modulo by zero	Error observed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 04: Traffic Violation Penalty Calculator

1. Aim

To develop and examine a Python program for traffic violation penalty calculator, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

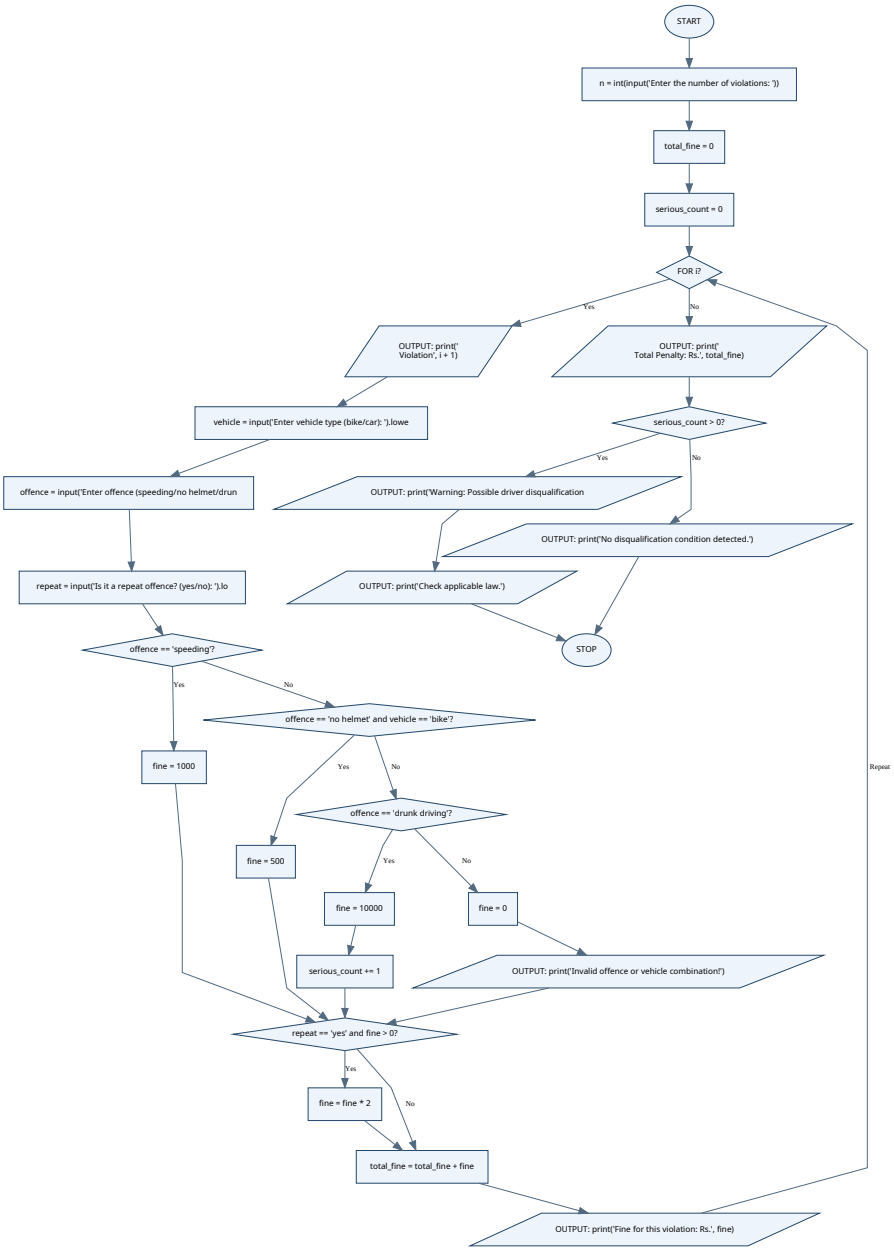
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
n = int(input('Enter the number of violations: '))
total_fine = 0
serious_count = 0
FOR i
  EXECUTE loop body
END LOOP
OUTPUT print('\nTotal Penalty: Rs.', total_fine)
IF serious_count > 0
  EXECUTE conditional branch
ELSE (if applicable)
END IF
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Traffic Penalty Calculator
n = int(input("Enter the number of violations: "))
total_fine = 0
serious_count = 0
for i in range(n):
    print("\nViolation", i + 1)
    vehicle = input("Enter vehicle type (bike/car): ").lower()
    offence = input(
        "Enter offence (speeding/no helmet/drunk driving): "
    ).lower()
    repeat = input("Is it a repeat offence? (yes/no): ").lower()
    if offence == "speeding":
        fine = 1000
    elif offence == "no helmet" and vehicle == "bike":
        fine = 500
    elif offence == "drunk driving":
        fine = 10000
        serious_count += 1
    else:
        fine = 0
        print("Invalid offence or vehicle combination!")
    if repeat == "yes" and fine > 0:
        fine = fine * 2
    total_fine = total_fine + fine
    print("Fine for this violation: Rs.", fine)
print("\nTotal Penalty: Rs.", total_fine)
if serious_count > 0:
    print("Warning: Possible driver disqualification.")
    print("Check applicable law.")
else:
    print("No disqualification condition detected.")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['1', 'car', 'speeding', 'no']

```
Enter the number of violations:
Violation 1
Enter vehicle type (bike/car): Enter offence (speeding/no helmet/drunk driving): Is it a repeat
offence? (yes/no): Fine for this violation: Rs. 1000
Total Penalty: Rs. 1000
No disqualification condition detected.
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 1, car, speeding, no	Defined by original program logic; see actual execution	Total Penalty: Rs. 1000; No disqualification condition detected.	Executed

Test Case	Input	Expected Output	Actual Output	Result
TC02	Boundary / alternate input 1, bike, no helmet, yes	Defined by original program logic; see actual execution	Total Penalty: Rs. 1000; No disqualification condition detected.	Executed
TC03	Invalid / edge input 1, car, unknown, no	Defined by original program logic; see actual execution	Total Penalty: Rs. 0; No disqualification condition detected.	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 05: Campus Event Financial Planning

1. Aim

To develop and examine a Python program for campus event financial planning, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

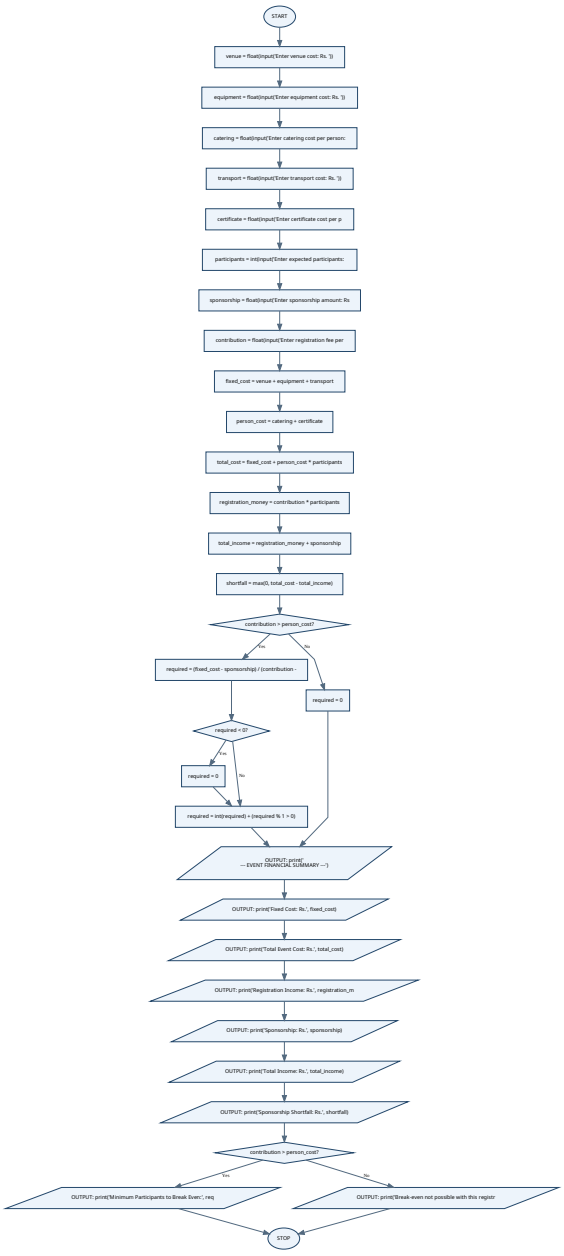
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
venue = float(input('Enter venue cost: Rs. '))
equipment = float(input('Enter equipment cost: Rs. '))
catering = float(input('Enter catering cost per person: Rs. '))
transport = float(input('Enter transport cost: Rs. '))
certificate = float(input('Enter certificate cost per person: Rs. '))
participants = int(input('Enter expected participants: '))
sponsorship = float(input('Enter sponsorship amount: Rs. '))
contribution = float(input('Enter registration fee per person: Rs. '))
fixed_cost = venue + equipment + transport
person_cost = catering + certificate
total_cost = fixed_cost + person_cost * participants
registration_money = contribution * participants
total_income = registration_money + sponsorship
shortfall = max(0, total_cost - total_income)
IF contribution > person_cost
  EXECUTE conditional branch
ELSE (if applicable)
END IF
OUTPUT print('\n--- EVENT FINANCIAL SUMMARY ---')
OUTPUT print('Fixed Cost: Rs.', fixed_cost)
OUTPUT print('Total Event Cost: Rs.', total_cost)
OUTPUT print('Registration Income: Rs.', registration_money)
OUTPUT print('Sponsorship: Rs.', sponsorship)
OUTPUT print('Total Income: Rs.', total_income)
OUTPUT print('Sponsorship Shortfall: Rs.', shortfall)
IF contribution > person_cost
  EXECUTE conditional branch
ELSE (if applicable)
END IF
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Campus Event Financial Planning
venue = float(input("Enter venue cost: Rs. "))
equipment = float(input("Enter equipment cost: Rs. "))
catering = float(input("Enter catering cost per person: Rs. "))
transport = float(input("Enter transport cost: Rs. "))
certificate = float(input("Enter certificate cost per person: Rs. "))
participants = int(input("Enter expected participants: "))
sponsorship = float(input("Enter sponsorship amount: Rs. "))
contribution = float(input("Enter registration fee per person: Rs. "))
fixed_cost = venue + equipment + transport
person_cost = catering + certificate
total_cost = fixed_cost + (person_cost * participants)
registration_money = contribution * participants
total_income = registration_money + sponsorship
shortfall = max(0, total_cost - total_income)
if contribution > person_cost:
    required = (fixed_cost - sponsorship) / (contribution - person_cost)
    if required < 0:
        required = 0
    required = int(required) + (required % 1 > 0)
else:
    required = 0
print("\n--- EVENT FINANCIAL SUMMARY ---")
print("Fixed Cost: Rs.", fixed_cost)
print("Total Event Cost: Rs.", total_cost)
print("Registration Income: Rs.", registration_money)
print("Sponsorship: Rs.", sponsorship)
print("Total Income: Rs.", total_income)
print("Sponsorship Shortfall: Rs.", shortfall)
if contribution > person_cost:
    print("Minimum Participants to Break Even:", required)
```

```
else:
    print("Break-even not possible with this registration fee.")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['10000', '5000', '200', '3000', '50', '100', '10000', '500']

```
Enter venue cost: Rs. Enter equipment cost: Rs. Enter catering cost per person: Rs. Enter transport
cost: Rs. Enter certificate cost per person: Rs. Enter expected participants: Enter sponsorship
amount: Rs. Enter registration fee per person: Rs.
--- EVENT FINANCIAL SUMMARY ---
Fixed Cost: Rs. 18000.0
Total Event Cost: Rs. 43000.0
Registration Income: Rs. 50000.0
Sponsorship: Rs. 10000.0
Total Income: Rs. 60000.0
Sponsorship Shortfall: Rs. 0
Minimum Participants to Break Even: 32
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 10000, 5000, 200, 3000, 50, 100, 10000, 500	Defined by original program logic; see actual execution	Sponsorship Shortfall: Rs. 0; Minimum Participants to Break Even: 32	Executed
TC02	Boundary / alternate input 10000, 5000, 300, 2000, 50, 50, 5000, 300	Defined by original program logic; see actual execution	Sponsorship Shortfall: Rs. 14500.0; Break-even not possible with this registration fee.	Executed
TC03	Invalid / edge input 0, 0, 0, 0, 0, 0, 0, 0	Defined by original program logic; see actual execution	Sponsorship Shortfall: Rs. 0; Break-even not possible with this registration fee.	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 06: Employee Payroll and Deduction Analysis

1. Aim

To develop and examine a Python program for employee payroll and deduction analysis, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

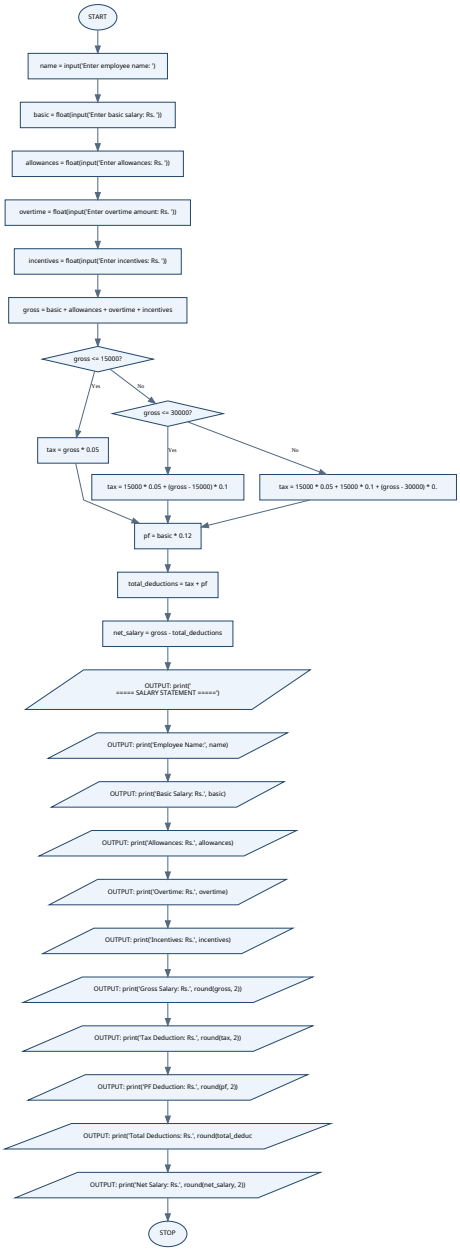
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
name = input('Enter employee name: ')
basic = float(input('Enter basic salary: Rs. '))
allowances = float(input('Enter allowances: Rs. '))
overtime = float(input('Enter overtime amount: Rs. '))
incentives = float(input('Enter incentives: Rs. '))
gross = basic + allowances + overtime + incentives
IF gross <= 15000
  EXECUTE conditional branch
ELSE (if applicable)
END IF
pf = basic * 0.12
total_deductions = tax + pf
net_salary = gross - total_deductions
OUTPUT print('\n===== SALARY STATEMENT =====')
OUTPUT print('Employee Name:', name)
OUTPUT print('Basic Salary: Rs.', basic)
OUTPUT print('Allowances: Rs.', allowances)
OUTPUT print('Overtime: Rs.', overtime)
OUTPUT print('Incentives: Rs.', incentives)
OUTPUT print('Gross Salary: Rs.', round(gross, 2))
OUTPUT print('Tax Deduction: Rs.', round(tax, 2))
OUTPUT print('PF Deduction: Rs.', round(pf, 2))
OUTPUT print('Total Deductions: Rs.', round(total_deductions, 2))
OUTPUT print('Net Salary: Rs.', round(net_salary, 2))
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Employee Payroll and Deduction Analysis
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: Rs. "))
allowances = float(input("Enter allowances: Rs. "))
overtime = float(input("Enter overtime amount: Rs. "))
incentives = float(input("Enter incentives: Rs. "))
gross = basic + allowances + overtime + incentives
if gross <= 15000:
    tax = gross * 0.05
elif gross <= 30000:
    tax = (15000 * 0.05) + ((gross - 15000) * 0.10)
else:
    tax = (15000 * 0.05) + (15000 * 0.10) + ((gross - 30000) * 0.15)
pf = basic * 0.12
total_deductions = tax + pf
net_salary = gross - total_deductions
print("\n===== SALARY STATEMENT =====")
print("Employee Name:", name)
print("Basic Salary: Rs.", basic)
print("Allowances: Rs.", allowances)
print("Overtime: Rs.", overtime)
print("Incentives: Rs.", incentives)
print("Gross Salary: Rs.", round(gross, 2))
print("Tax Deduction: Rs.", round(tax, 2))
print("PF Deduction: Rs.", round(pf, 2))
print("Total Deductions: Rs.", round(total_deductions, 2))
print("Net Salary: Rs.", round(net_salary, 2))
```

7. Output (Executed Sample)

Sample input values (in entry order): ['Ravi', '10000', '2000', '1000', '500']

```
Enter employee name: Enter basic salary: Rs. Enter allowances: Rs. Enter overtime amount: Rs. Enter
incentives: Rs.
===== SALARY STATEMENT =====
Employee Name: Ravi
Basic Salary: Rs. 10000.0
```

```
Allowances: Rs. 2000.0
Overtime: Rs. 1000.0
Incentives: Rs. 500.0
Gross Salary: Rs. 13500.0
Tax Deduction: Rs. 675.0
PF Deduction: Rs. 1200.0
Total Deductions: Rs. 1875.0
Net Salary: Rs. 11625.0
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input Ravi, 10000, 2000, 1000, 500	Defined by original program logic; see actual execution	Total Deductions: Rs. 1875.0; Net Salary: Rs. 11625.0	Executed
TC02	Boundary / alternate input Suresh, 25000, 5000, 2000, 3000	Defined by original program logic; see actual execution	Total Deductions: Rs. 6000.0; Net Salary: Rs. 29000.0	Executed
TC03	Invalid / edge input Test, 0, 0, 0, 0	Defined by original program logic; see actual execution	Total Deductions: Rs. 0.0; Net Salary: Rs. 0.0	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 07: Campus Festival Registration and Financial Audit

1. Aim

To develop and examine a Python program for campus festival registration and financial audit, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

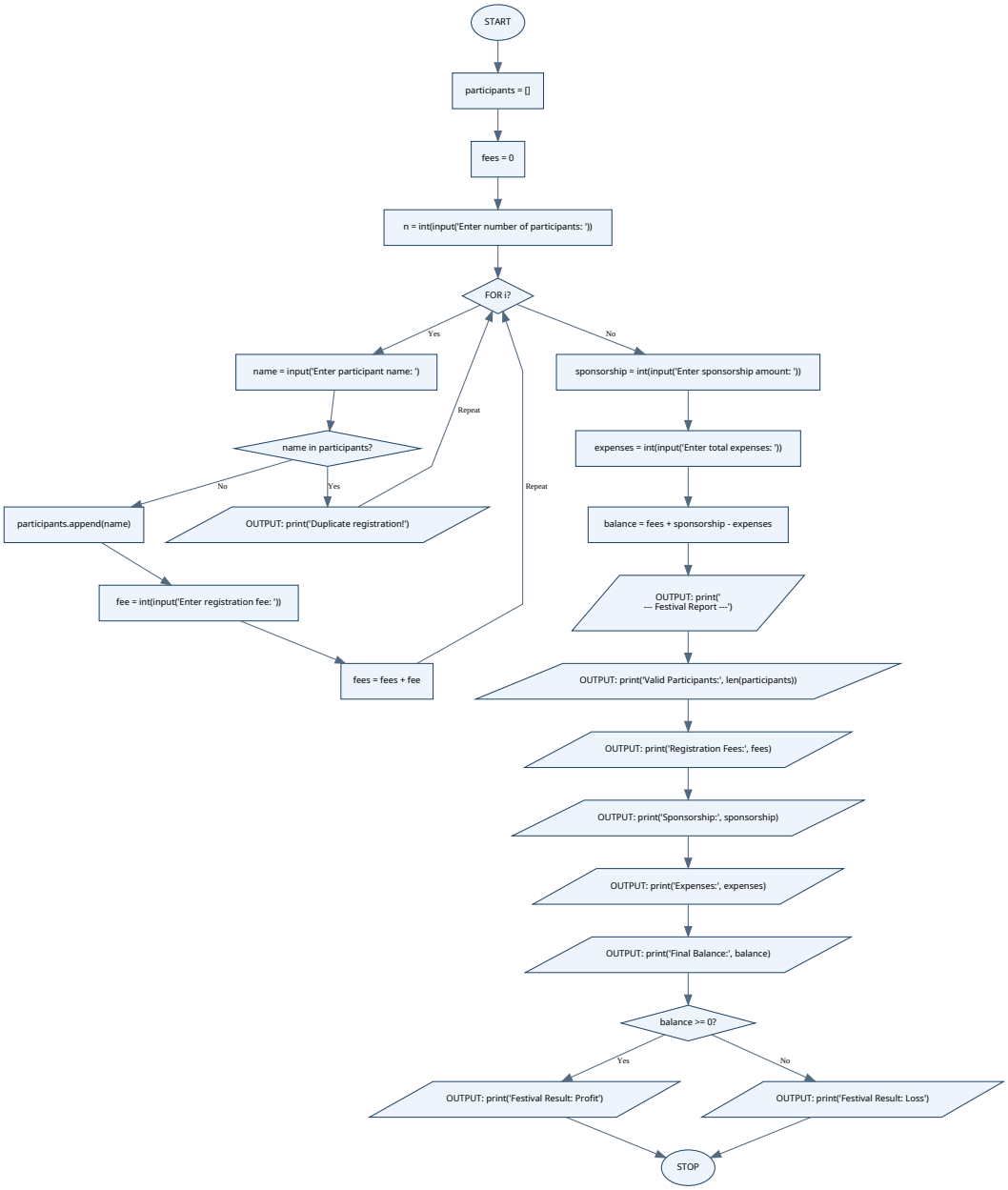
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
participants = []
fees = 0
n = int(input('Enter number of participants: '))
FOR i
  EXECUTE loop body
END LOOP
sponsorship = int(input('Enter sponsorship amount: '))
expenses = int(input('Enter total expenses: '))
balance = fees + sponsorship - expenses
OUTPUT print('\n--- Festival Report ---')
OUTPUT print('Valid Participants:', len(participants))
OUTPUT print('Registration Fees:', fees)
OUTPUT print('Sponsorship:', sponsorship)
OUTPUT print('Expenses:', expenses)
OUTPUT print('Final Balance:', balance)
IF balance >= 0
  EXECUTE conditional branch
ELSE (if applicable)
END IF
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Campus Festival Registration and Financial Audit
participants = []
fees = 0
n = int(input("Enter number of participants: "))
for i in range(n):
    name = input("Enter participant name: ")
    if name in participants:
        print("Duplicate registration!")
    else:
        participants.append(name)
        fee = int(input("Enter registration fee: "))
        fees = fees + fee
sponsorship = int(input("Enter sponsorship amount: "))
expenses = int(input("Enter total expenses: "))
balance = fees + sponsorship - expenses
print("\n--- Festival Report ---")
print("Valid Participants:", len(participants))
print("Registration Fees:", fees)
print("Sponsorship:", sponsorship)
print("Expenses:", expenses)
print("Final Balance:", balance)
if balance >= 0:
    print("Festival Result: Profit")
else:
    print("Festival Result: Loss")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['3', 'Ravi', '500', 'Sita', '500', 'Kumar', '500', '2000', '2500']

```
Enter number of participants: Enter participant name: Enter registration fee: Enter participant
name: Enter registration fee: Enter participant name: Enter registration fee: Enter sponsorship
amount: Enter total expenses:
--- Festival Report ---
Valid Participants: 3
Registration Fees: 1500
Sponsorship: 2000
Expenses: 2500
Final Balance: 1000
Festival Result: Profit
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 3, Ravi, 500, Sita, 500, Kumar, 500, 2000, 2500	Defined by original program logic; see actual execution	Final Balance: 1000; Festival Result: Profit	Executed
TC02	Boundary / alternate input 3, Ravi, 500, Sita, 500, Ravi, 500, 2000	Defined by original program logic; see actual execution	Final Balance: -500; Festival Result: Loss	Executed
TC03	Invalid / edge input 0, 0, 0	Defined by original program logic; see actual execution	Final Balance: 0; Festival Result: Profit	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 08: Water Distribution Fairness Analysis

1. Aim

To develop and examine a Python program for water distribution fairness analysis, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

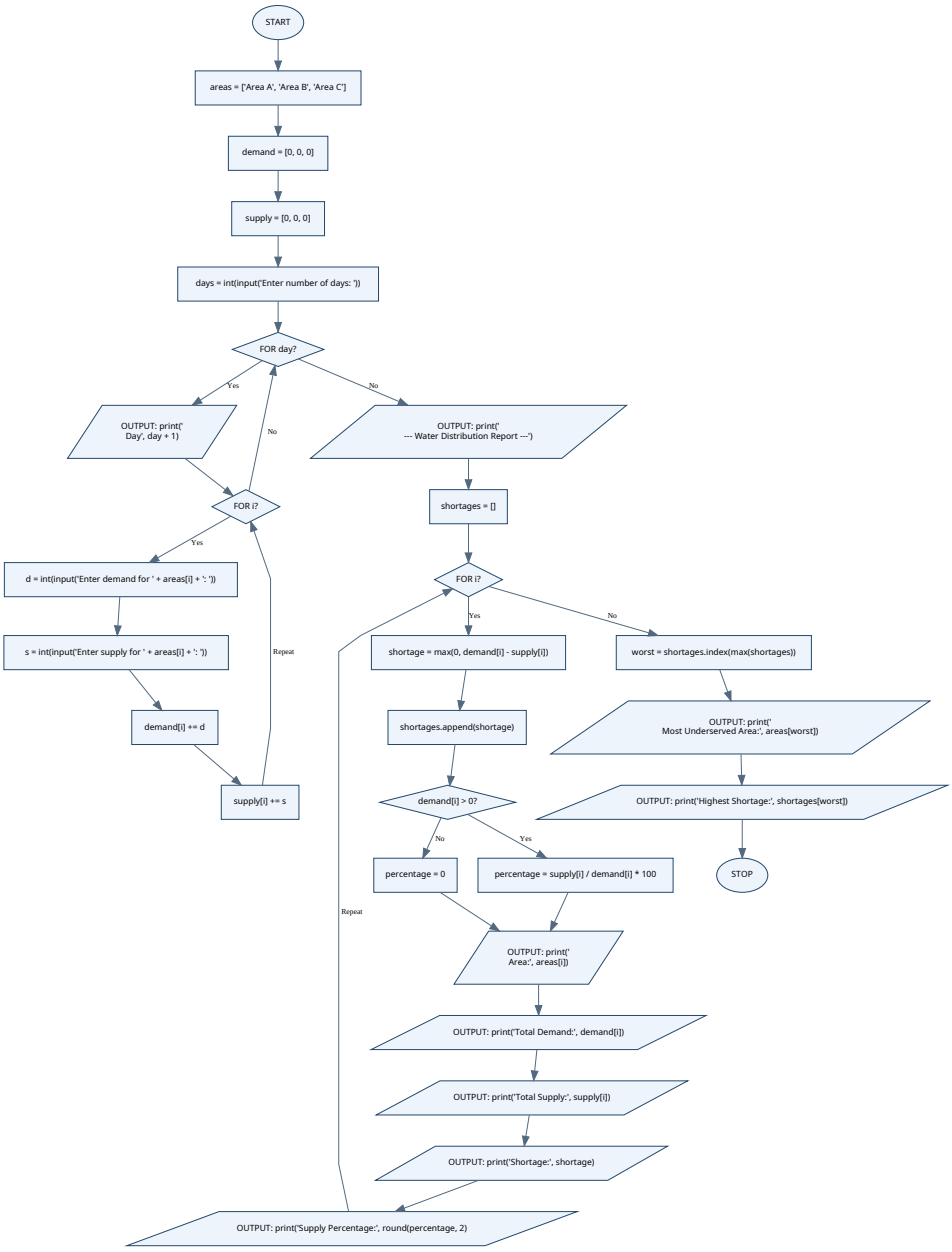
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
areas = ['Area A', 'Area B', 'Area C']
demand = [0, 0, 0]
supply = [0, 0, 0]
days = int(input('Enter number of days: '))
FOR day
  EXECUTE loop body
END LOOP
OUTPUT print('\n--- Water Distribution Report ---')
shortages = []
FOR i
  EXECUTE loop body
END LOOP
worst = shortages.index(max(shortages))
OUTPUT print('\nMost Underserved Area:', areas[worst])
OUTPUT print('Highest Shortage:', shortages[worst])
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Water Distribution Fairness Analysis
areas = ["Area A", "Area B", "Area C"]
demand = [0, 0, 0]
supply = [0, 0, 0]
days = int(input("Enter number of days: "))
for day in range(days):
    print("\nDay", day + 1)
    for i in range(3):
        d = int(input("Enter demand for " + areas[i] + ": "))
        s = int(input("Enter supply for " + areas[i] + ": "))
        demand[i] += d
        supply[i] += s
print("\n--- Water Distribution Report ---")
shortages = []
for i in range(3):
    shortage = max(0, demand[i] - supply[i])
    shortages.append(shortage)
    if demand[i] > 0:
        percentage = supply[i] / demand[i] * 100
    else:
        percentage = 0
    print("\nArea:", areas[i])
    print("Total Demand:", demand[i])
    print("Total Supply:", supply[i])
    print("Shortage:", shortage)
    print("Supply Percentage:", round(percentage, 2), "%")
worst = shortages.index(max(shortages))
print("\nMost Underserved Area:", areas[worst])
print("Highest Shortage:", shortages[worst])
```

7. Output (Executed Sample)

Sample input values (in entry order): ['1', '1000', '800', '1200', '900', '800', '700']

```
Enter number of days:
Day 1
Enter demand for Area A: Enter supply for Area A: Enter demand for Area B: Enter supply for Area B:
Enter demand for Area C: Enter supply for Area C:
--- Water Distribution Report ---
Area: Area A
Total Demand: 1000
Total Supply: 800
Shortage: 200
Supply Percentage: 80.0 %
Area: Area B
Total Demand: 1200
```

```
Total Supply: 900
Shortage: 300
Supply Percentage: 75.0 %
Area: Area C
Total Demand: 800
Total Supply: 700
Shortage: 100
Supply Percentage: 87.5 %
Most Underserved Area: Area B
Highest Shortage: 300
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 1, 1000, 800, 1200, 900, 800, 700	Defined by original program logic; see actual execution	Most Underserved Area: Area B; Highest Shortage: 300	Executed
TC02	Boundary / alternate input 1, 0, 0, 0, 0, 0, 0	Defined by original program logic; see actual execution	Most Underserved Area: Area A; Highest Shortage: 0	Executed
TC03	Invalid / edge input 0	Defined by original program logic; see actual execution	Most Underserved Area: Area A; Highest Shortage: 0	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 09: Mobile Plan Cost Optimization System

1. Aim

To develop and examine a Python program for mobile plan cost optimization system, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

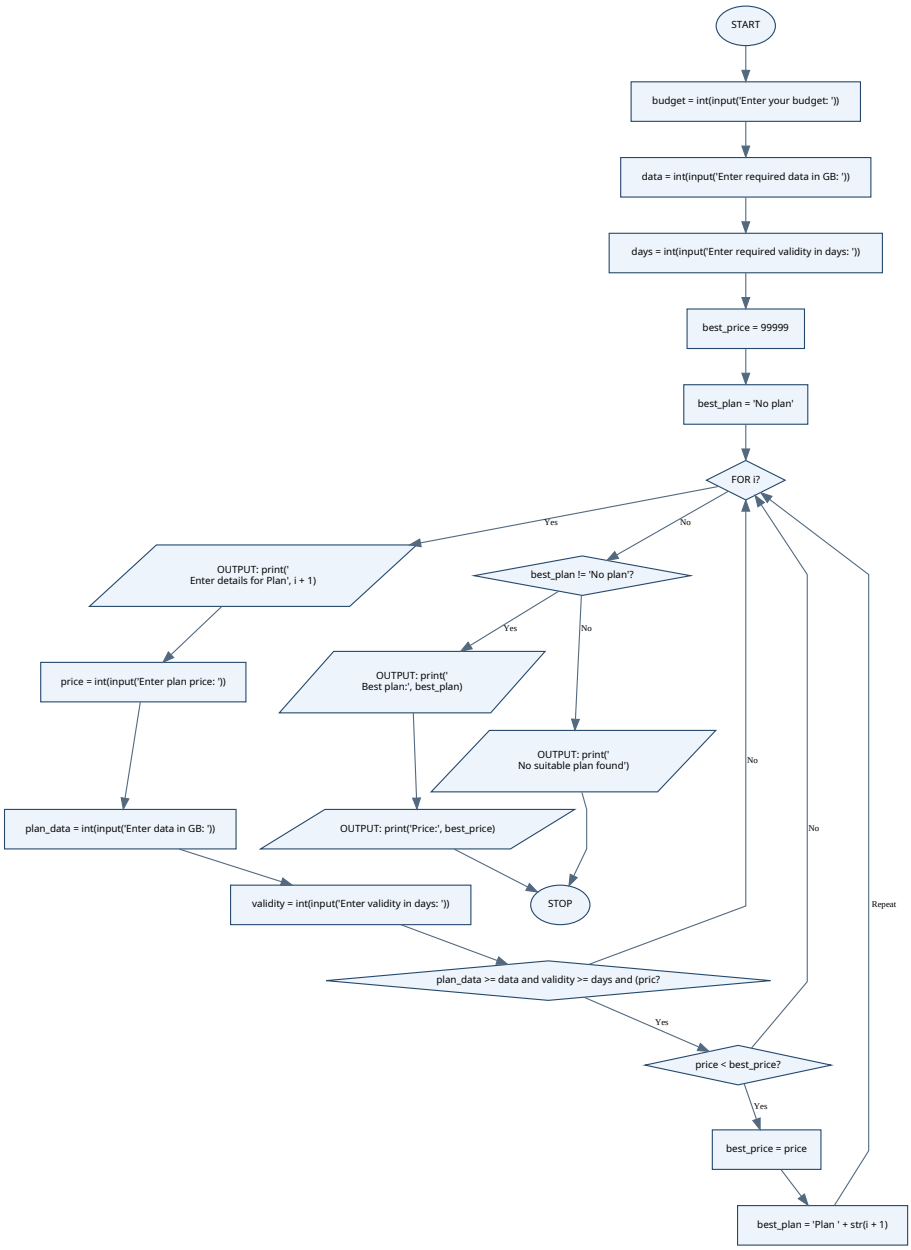
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
budget = int(input('Enter your budget: '))
data = int(input('Enter required data in GB: '))
days = int(input('Enter required validity in days: '))
best_price = 99999
best_plan = 'No plan'
FOR i
  EXECUTE loop body
END LOOP
IF best_plan != 'No plan'
  EXECUTE conditional branch
ELSE (if applicable)
END IF
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
budget = int(input("Enter your budget: "))
data = int(input("Enter required data in GB: "))
days = int(input("Enter required validity in days: "))
best_price = 99999
best_plan = "No plan"
for i in range(3):
    print("\nEnter details for Plan", i + 1)
    price = int(input("Enter plan price: "))
    plan_data = int(input("Enter data in GB: "))
    validity = int(input("Enter validity in days: "))
    if plan_data >= data and validity >= days and price <= budget:
        if price < best_price:
            best_price = price
            best_plan = "Plan " + str(i + 1)
if best_plan != "No plan":
    print("\nBest plan:", best_plan)
    print("Price:", best_price)
else:
    print("\nNo suitable plan found")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['300', '2', '28', '250', '2', '28', '199', '1', '28', '350', '5', '28']

```
Enter your budget: Enter required data in GB: Enter required validity in days:
Enter details for Plan 1
Enter plan price: Enter data in GB: Enter validity in days:
Enter details for Plan 2
Enter plan price: Enter data in GB: Enter validity in days:
Enter details for Plan 3
Enter plan price: Enter data in GB: Enter validity in days:
Best plan: Plan 1
Price: 250
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 300, 2, 28, 250, 2, 28, 199, 1, 28, 350, 5, 28	Defined by original program logic; see actual execution	Best plan: Plan 1; Price: 250	Executed
TC02	Boundary / alternate input 100, 10, 90, 200, 1, 10, 300, 2, 20, 400, 3, 30	Defined by original program logic; see actual execution	Enter plan price: Enter data in GB: Enter validity in days: ; No suitable plan found	Executed
TC03	Invalid / edge input 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	Defined by original program logic; see actual execution	Best plan: Plan 1; Price: 0	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.

EXPERIMENT 10: Food Waste Monitoring System

1. Aim

To develop and examine a Python program for food waste monitoring system, and verify its behavior using representative inputs.

2. Procedure

Review the problem statement and identify the required inputs and outputs.

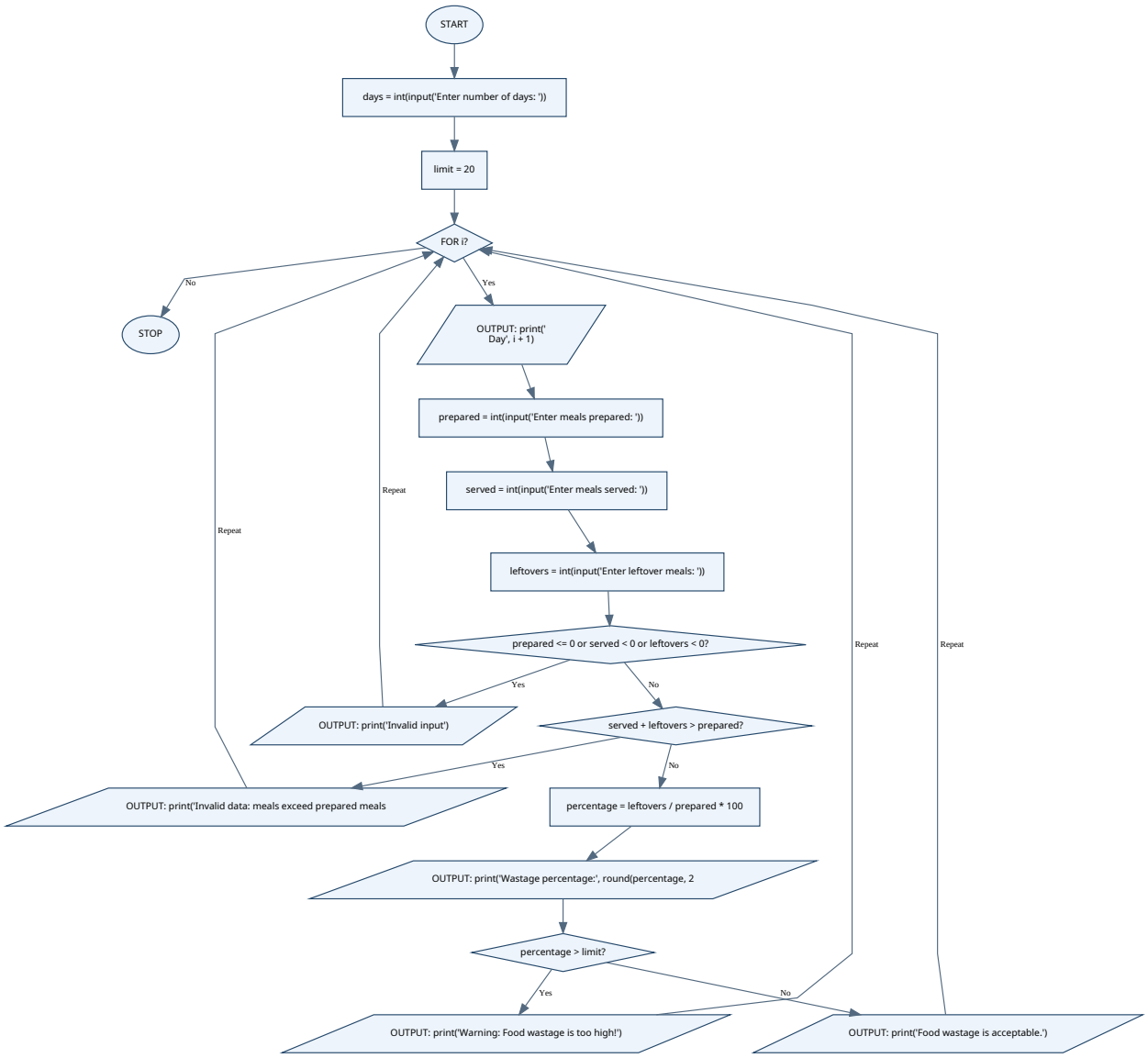
Enter the supplied source code in a Python 3 environment without altering its logic.

Run the program and provide the specified input values.

Observe the displayed results and any errors.

Compare the execution with the documented test cases.

3. Flowchart (Standard Symbols)



4. Pseudocode

```
START
days = int(input('Enter number of days: '))
limit = 20
FOR i
```

```
  EXECUTE loop body
END LOOP
STOP
```

5. Algorithm

START. Initialize the variables and read inputs as specified in the program. Execute the calculations, conditional branches and iterations in the order shown in the flowchart. Display the computed values or messages. STOP.

6. Source Code (from uploaded PDF)

```
# Food Waste Monitoring System
days = int(input("Enter number of days: "))
limit = 20
for i in range(days):
    print("\nDay", i + 1)
    prepared = int(input("Enter meals prepared: "))
    served = int(input("Enter meals served: "))
    leftovers = int(input("Enter leftover meals: "))
    if prepared <= 0 or served < 0 or leftovers < 0:
        print("Invalid input")
    elif served + leftovers > prepared:
        print("Invalid data: meals exceed prepared meals")
    else:
        percentage = (leftovers / prepared) * 100
        print("Wastage percentage:", round(percentage, 2), "%")
        if percentage > limit:
            print("Warning: Food wastage is too high!")
        else:
            print("Food wastage is acceptable.")
```

7. Output (Executed Sample)

Sample input values (in entry order): ['1', '100', '80', '20']

```
Enter number of days:
Day 1
Enter meals prepared: Enter meals served: Enter leftover meals: Wastage percentage: 20.0 %
Food wastage is acceptable.
```

8. Test Cases

Test Case	Input	Expected Output	Actual Output	Result
TC01	Typical valid input 1, 100, 80, 20	Defined by original program logic; see actual execution	Enter meals prepared: Enter meals served: Enter leftover meals: Wastage percentage: 20.0 %; Food wastage is ac	Executed
TC02	Boundary / alternate input 1, 100, 70, 30	Defined by original program logic; see actual execution	Enter meals prepared: Enter meals served: Enter leftover meals: Wastage percentage: 30.0 %; Warning: Food wast	Executed
TC03	Invalid / edge input 1, 0, 0, 0	Defined by original program logic; see actual execution	Day 1; Enter meals prepared: Enter meals served: Enter leftover meals: Invalid input	Executed

9. Result

The program was tested with the inputs shown above. Any execution errors are recorded rather than silently corrected.